

```

// FURNACE v5.3.1 review suite – 66 checks.
Run: pnpm add -D tsx @noble/hashes && pnpm
tsx furnace.review.test.ts
// Needs hash-registry.json beside it (or
REGISTRY=path). Exits with a count; wire
into CI as a gate.
import cfg, {
  validateConfig, heatAgeAfterWithdrawal,
  cooledAgeAfterWithdrawal, deedHeatAge,
  create3Address, isValidEvmAddress,
  isValidSolanaPubkey, epochAllocation,
  totalMintForEpoch, heatMultiplier,
} from './furnace.config';
import { readFileSync } from 'node:fs';

let pass = 0, fail = 0;
const t = (name: string, ok: boolean,
detail = '') => { ok ? pass++ : fail++;
console.log(`${ok ? '✓' : '✗'}
${name}${detail ? ' ' + detail : ''}`); };
const near = (a: number, b: number, eps =
1e-6) => Math.abs(a - b) < eps;
const clone = () =>
JSON.parse(JSON.stringify(cfg));

// — Baseline


---




---


const baseline = validateConfig(cfg);
console.log(`\nBaseline: ${baseline.length}

```

```

errors`); baseline.forEach(e =>
console.log('  ·', e));
const isDeployBlocker = (e: string) =>
/deployer|saltNonce|signers|threshold|guard
ian|dvns|treasury\\.addresses/.test(e);
t('baseline contains ONLY deploy blockers',
baseline.every(isDeployBlocker));

```

```

// — 28 reviewer mutations

```

```

console.log('\n— Reviewer mutations (each
must add ≥1 rejection) —');
const first = (c: any) =>
c.deployment.evm.contracts[Object.keys(c.de
ployment.evm.contracts)[0]];
const mutations: [string, (c: any) => void]
[] = [
  ['paid-claim', c => c.forge.fees.claimPct
= 1],
  ['emergency-above-ceiling', c =>
c.forge.emergencyExit.feePct = 99],
  ['total-reset-cooling', c =>
c.forge.heat.withdrawCooling = 'reset'],
  ['non-inheriting-stoke', c =>
c.forge.heat.compoundInheritsAge = false],
  ['floor-2000', c =>
c.emissions.base.floorPerDay = '2000'],
  ['step-emissions', c =>
c.emissions.base.shape = 'step'],
  ['ordinary-0-100', c => {
c.emissions.split.flatPct = 0;
c.emissions.split.heatPct = 100; }],
  ['conviction-90-percent', c =>

```

```
c.emissions.convictionBucketPct = 90],
  ['first-withdrawal-tier-7-hours', c =>
c.cooldowns.withdraw.tiers[0].seconds =
25200],
  ['fresh-lot-on-lapse', c =>
c.forge.cancelReturnsAsFreshLot = true],
  ['reserve-chunk-50-percent', c =>
c.fees.burnLane.maxChunkPctOfReserve = 50],
  ['price-band-50-percent', c =>
c.fees.burnLane.avgPriceBandBps = 5000],
  ['disable-route-allowlist', c =>
c.fees.burnLane.allowlistedRoutersOnly =
false],
  ['disable-unsafe-deferral', c =>
c.fees.burnLane.deferIfUnsafe = false],
  ['kindling-100-percent-permanent', c => {
c.kindling.lpSplit.depositorDeedsPct = 0;
c.kindling.lpSplit.burnPct = 100; }],
  ['kindling-not-atomic', c =>
c.kindling.atomicPoolInit = false],
  ['fissure-fee-4-percent', c =>
c.fissures.feePct = 4],
  ['genesis-seed-disagrees', c =>
c.token.allocations.kindlingPool =
'500000'],
  ['raise-both-ember-caps', c => {
c.token.maxSupply = '42000000';
c.emissions.supplyCeiling.hardCap =
'42000000'; }],
  ['raise-ash-cap-and-promises', c => {
c.shareToken.maxSupply = '80000';
c.shareToken.allocations.forge = '38000';
}],
  ['native-router-wrong-length', c =>
```

```

c.chains[0].externalDex.router =
'0x7a250d5630B4cF539739dF2C5dAcB4c8440c05']
,
  ['same-prefix-wrong-derived-addr', c => {
const a = first(c); a.address =
a.address.slice(0, 6) +
'abcdef0123456789abcdef0123456789abcd'; }],
  ['salt-nonhex-entropy', c => { const a =
first(c); a.salt = a.salt.slice(0, 44) +
'zz'.repeat(11); }],
  ['transfer-outside-market', c =>
c.forge.deeds.marketOnlyTransfers = false],
  ['list-with-pending-request', c =>
c.forge.deeds.blockListingWithPending =
false],
  ['one-second-auction', c =>
c.forge.deeds.auctionHours = 1 / 3600],
  ['nan-buyback-slippage', c =>
c.fees.burnLane.slippageCapBps = NaN],
  ['no-shape-sqrt', c => c.forge.heat.shape
= 'linear'],
];
const base = new Set(baseline);
const run = (mut: (c: any) => void) => {
const c = clone(); mut(c); try { return
validateConfig(c).filter(e =>
!base.has(e)); } catch (e) { return ['THREW
${e}`']; } };
let caught = 0;
for (const [id, m] of mutations) { const
added = run(m); if (added.length) caught++;
t(id, added.length > 0, added[0] ?? 'NOT
CAUGHT'); }
console.log(`→

```

```

    ${caught}/${mutations.length} caught`);
    // NB: JSON clone turns NaN into null –
    test NaN on a structured clone too
    { const c = structuredClone(cfg) as any;
      c.fees.burnLane.slippageCapBps = NaN;
      t('nan survives structuredClone and is
        rejected', validateConfig(c).some(e =>
        /finite/.test(e))); }

    // — Reviewer's 9 positive controls must
    still fire —————
    console.log(`\n— Positive controls —`);
    const controls: [string, (c: any) => void]
    [] = [
      ['deposit-fee-2', c =>
      c.forge.fees.depositPct = 2],
      ['keeper-tip-2', c =>
      c.forge.autoStoke.tip.maxPct = 2],
      ['keeper-hourly', c =>
      c.forge.autoStoke.minInterval.minHours =
      1],
      ['ash-kindling-budget-drift', c =>
      c.kindling.ashBonusPool = '2000'],
      ['unmined-ember', c => { const a =
      c.deployment.evm.contracts.EmberToken;
      a.salt = null; a.address = null; }],
      ['twap-one-second', c =>
      c.fees.burnLane.twapSeconds = 1],
      ['all-vent-to-treasury', c => c.fees.vent
      = { burnPct: 0, liquidityPct: 0, hearthPct:
      0, treasuryPct: 100 }],
      ['weighted-average-deposits', c =>
      c.forge.heat.depositDilution =
      'weightedAverage'],

```

```

    ['cast-not-source-lot', c =>
c.forge.casting.castLot = 'rollingLot'],
];
for (const [id, m] of controls) { const
added = run(m); t(id, added.length > 0,
added[0] ?? 'REGRESSED'); }

// — CREATE3: reproduce all 9 addresses
from hash-registry.json —————
console.log('\n— CREATE3 derivation vs
reviewer registry —');
const reg =
JSON.parse(readFileSync(process.env.REGISTR
Y ?? './hash-registry.json',
'utf8')).registry as { contract: string;
derived: string }[];
const map: Record<string, string> = {
FeeRouter: 'FeeRouter' };
const contracts =
cfg.deployment.evm.contracts as
Record<string, { salt: string; address:
string }>;
for (const r of reg) {
    const entry = contracts[r.contract];
    if (!entry) { t(`${r.contract} present in
config`, false); continue; }
    const d = create3Address(entry.salt,
cfg.deployment.evm.deployer,
cfg.deployment.evm.factory);
    t(`${r.contract.padEnd(11)} ${d}`,
d.toLowerCase() ===
r.derived.toLowerCase());
}

```

```
// — Address helpers



---



console.log('\n— Address helpers —');
t('Router02 published address valid',
  isValidEvmAddress('0x7a250d5630B4cF539739dF
2C5dAcB4c659F2488D'));
t('my earlier padded guess rejected',
  !isValidEvmAddress('0x7a250d5630B4cF539739d
F2C5dAcB4c8440c05e9'));
t('19-byte router rejected',
  !isValidEvmAddress('0x7a250d5630B4cF539739d
F2C5dAcB4c8440c05'));
t('Jupiter v6 is a 32-byte pubkey',
  isValidSolanaPubkey('JUP6LkbZbjS1jKKwapdHNy
74zcZ3tLUZoi5QNyVTaV4'));
t('base58-charset-but-wrong-length
rejected',
  !isValidSolanaPubkey('F1REtreasury596aBcDeF
gHiJkLmNoPqRsTuVwXyZ123'.slice(0, 20)));
t('base58 with invalid char (0) rejected',
  !isValidSolanaPubkey('0' +
'JUP6LkbZbjS1jKKwapdHNy74zcZ3tLUZoi5QNyVTaV
'));

// — §4.1 helper consistency



---



console.log('\n— Helpers (§4.1, §8) —');
t('heatAgeAfterWithdrawal == cooledAge at
730d/50%', heatAgeAfterWithdrawal(730, .5)
=== cooledAgeAfterWithdrawal(730, .5) &&
cooledAgeAfterWithdrawal(730, .5) ===
182.5);
```

```

t('deedHeatAge(730) = 292 (cap first)',
deedHeatAge(730) === 292);
t('deedHeatAge(100) = 80', deedHeatAge(100)
=== 80);
t('totalMintForEpoch shared headroom 100/50
@ 90 → 60/30', (() => { const r =
totalMintForEpoch(100, 50, 21_000_000 -
90); return near(r.base, 60) &&
near(r.ashfall, 30); })());
t('totalMintForEpoch rejects negative
base', (() => { try {
totalMintForEpoch(-100, 200, 20_999_990);
return false; } catch { return true; } })
());
t('heat continuous at every milestone ( $|\Delta|$ 
< 1e-6 across  $\pm 1e-7d$ )', [7, 30, 90, 180,
365].every(m => Math.abs(heatMultiplier(m +
1e-7) - heatMultiplier(m - 1e-7)) < 1e-6));

// — §6 Conviction oracle: reproduce
reviewer numbers —————
console.log('\n— Conviction allocator
($6) —');
const ce = epochAllocation(10_000, [{
stake: 10, heat: 3.65, sealBonus: 0.5 }, {
stake: 990, heat: 1, sealBonus: 0 }]);
t('sealer escrow = additive cap 30.6868',
near(ce.escrow[0], 30.686799805163172, 1e-
9), ce.escrow[0].toFixed(6));
t('literal cap (112.0) NOT used',
ce.escrow[0] < 50);
t('conservation  $\Sigma \text{ordinary} + \Sigma \text{escrow} = E$ ',
near(ce.ordinary.reduce((a, b) => a + b, 0)
+ ce.escrow.reduce((a, b) => a + b, 0),

```

```

10_000, 1e-6));
const empty = epochAllocation(10_000, [{
  stake: 1, heat: 1, sealBonus: 0 }, { stake:
  1, heat: 3, sealBonus: 0 }]);
t('empty bucket: equal stakes 1×/3× →
31.75% / 68.25%', near(empty.ordinary[0] /
10_000, 0.3175) && near(empty.ordinary[1] /
10_000, 0.6825));
t('empty bucket returns all of K',
near(empty.returned, 1000));
// property: 1,000 random cohorts conserve
budget and never exceed caps
let prop = true, seed = 42; const rnd = ()
=> (seed = (seed * 1103515245 + 12345) % 2
** 31) / 2 ** 31;
const bonuses = [0, 0, 0.15, 0.25, 0.35,
0.5];
for (let i = 0; i < 1000 && prop; i++) {
  const n = 1 + Math.floor(rnd() * 20);
  const ps = Array.from({ length: n }, ()
=> ({ stake: rnd() * 1000, heat: 1 + rnd()
* 3.15, sealBonus: bonuses[Math.floor(rnd()
* 6)] }));
  const E = 1 + rnd() * 50_000; const r =
epochAllocation(E, ps);
  const W = ps.reduce((a, p) => a + p.stake
* p.heat, 0); const L = 0.7 * 0.9 * E;
  const total = r.ordinary.reduce((a, b) =>
a + b, 0) + r.escrow.reduce((a, b) => a +
b, 0);
  if (!near(total, E, 1e-6 * E)) prop =
false;
  ps.forEach((p, j) => { if (r.escrow[j] >
(L * p.stake * p.sealBonus) / W + 1e-9)

```

```
prop = false; if (r.escrow[j] < 0 ||  
r.ordinary[j] < 0) prop = false; });  
    if (r.returned < -1e-9) prop = false;  
}  
t('1,000 seeded cohorts: conservation, caps  
bind, no negatives', prop);  
  
console.log(`\n== ${pass} passed, ${fail}  
failed ==`);
```